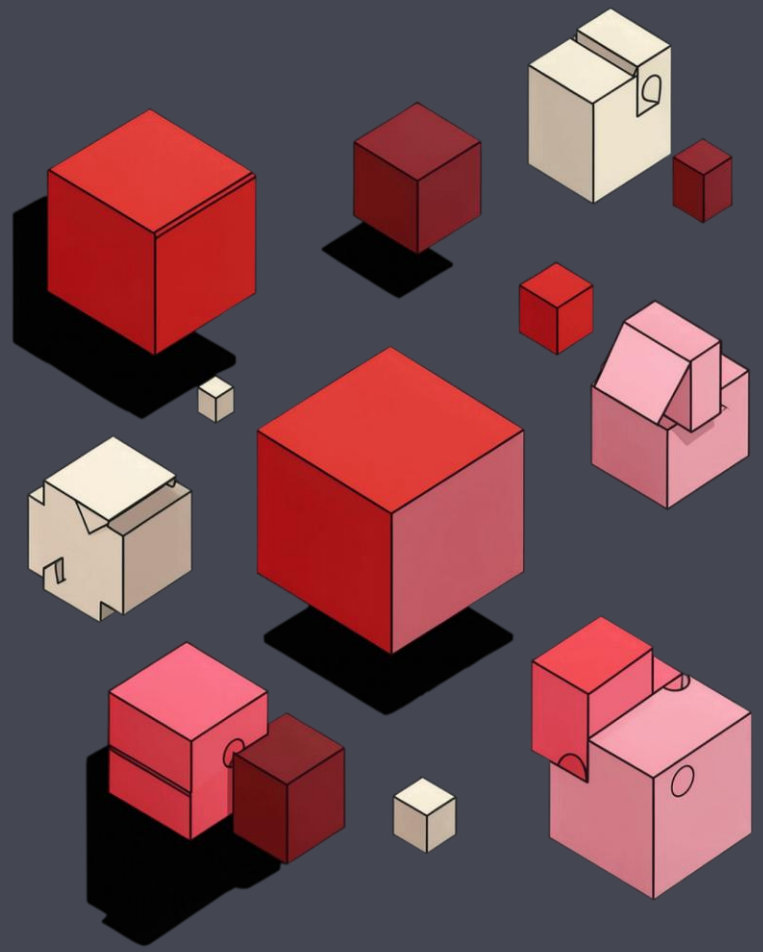
Programación Orientada a Entornos Multimedia

Semana 1: Gráficos por Computadora, WebGL y Primer Renderizado 3D

Gabriel Guerrero Arellano

UNIVERSIDAD COOPERATIVA DE COLOMBIA



Ruta de Aprendizaje: Programación de Entornos Multimedia

MOMENTO 1: Fundamentos 3D e Interactividad Básica



SEMANA 1: Fundamentos 3D y WebGL

Exploración del pipeline gráfico, ecosistema Node.js/Vite y el renderizado inicial de Escena, Cámara y Renderizador.



SEMANA 2: Geometrías y Materiales

Creación de mallas (Meshes), aplicación de colores, texturas y transformaciones de rotación, escala y posición.



SEMANA 3: Medios, Animación y Audio

Implementación del bucle de animación a 60 FPS, sincronización de video como textura y configuración de audio espacial.



SEMANA 4: Iluminación y Partículas

Manejo de luces ambientales, direccionales y tipo spat, jonto con la creación de sistemas de particulas para efectos de fuego o lluvia.

MOMENTO 2: Físicas, Control y Carga de Assets



SEMANA 5: Físicas 3D (Parte 1)

Introducción a simulaciones físicas, incluyendo la implementación de gravedad y detección de colisiones básicas.



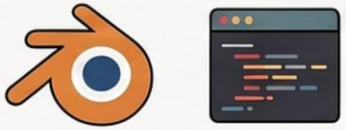
SEMANA 6: Físicas Avanzadas y Control

Gestión de múltiples objetos físicos y desarrollo de un personaje o entidad controlable por el usuario.



SEMANA 7: Importación de Assets

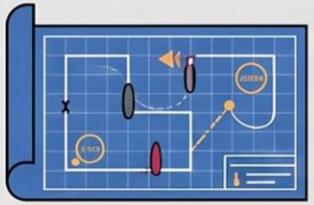
Carga de modelos externos (.gltf / .glb), uso de Environment Maps y aplicación de materiales tipo Matcap.



SEMANA 8: Integración con Blender

Configuración de escenarios en Blender, exportación eficiente de recursos y carga completa de la escena en el entorno Three.js.

MOMENTO 3: Desarrollo, Optimización y Proyecto Final



SEMANA 9: Configuración del Proyecto Integrador

Definición de mecánicas interactivas avanzadas como portales y activación de sonidos por zonas, incluyendo deporación de código.



SEMANA 10: Ensamblaje del Mundo Interactivo

Integración total de modelos de Blender, sistemas físicos, iluminación, partículas y controles de navegación.



SEMANA 11: Render Final y Sustentación

Optimización técnica del rendimiento para la web, demostración técnica en vivo y defensa profesional del proyecto.



SEMANA 12: Proyecto Integrador Final

Entrega y sustentación de una escena 3D interactiva funcional que unifica todos los conceptos aprendidos durante el curso.

Gemini Notebook

Ruta de la Clase

01

Semana 1

Primer Renderizado 3D

02

Concepto

WebGL y la "Trinidad" del 3D

03

Herramienta

VS Code, Live Server y Three.js (CDN)

04

Laboratorio

Creación de nuestro primer mundo y cubo 3D

05

Evidencia

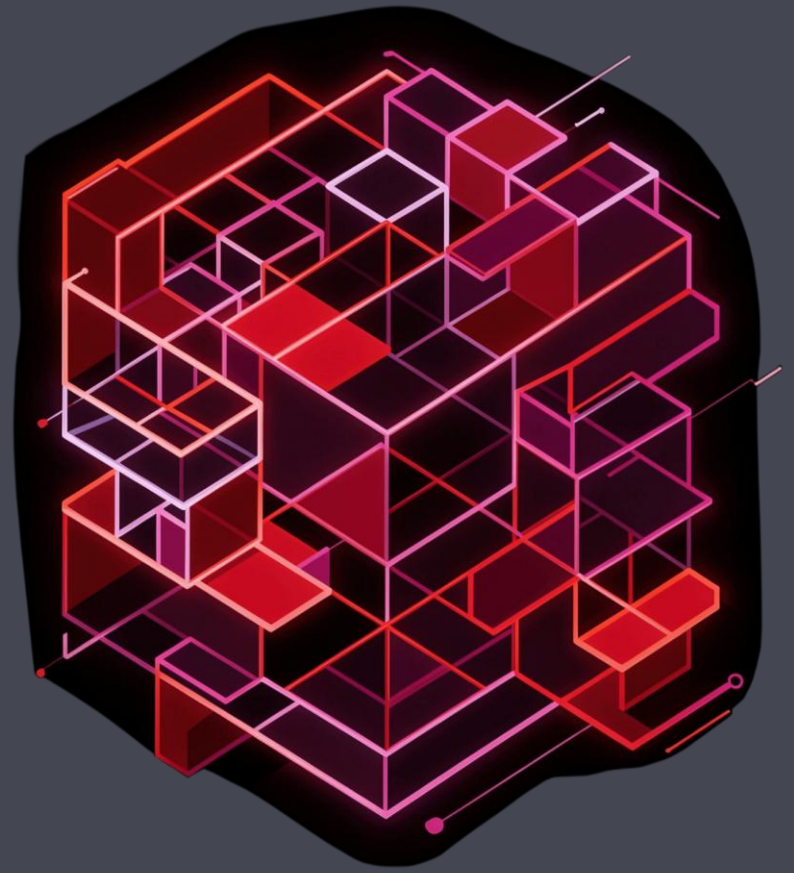
Captura del renderizado funcionando en el navegador

FASE 2

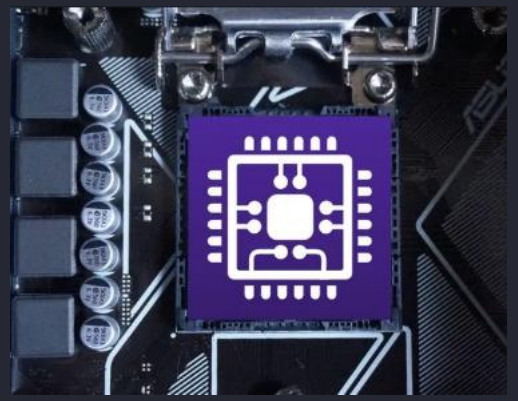
EL CONCEPTO

Fundamentos de WebGL y la arquitectura del mundo 3D

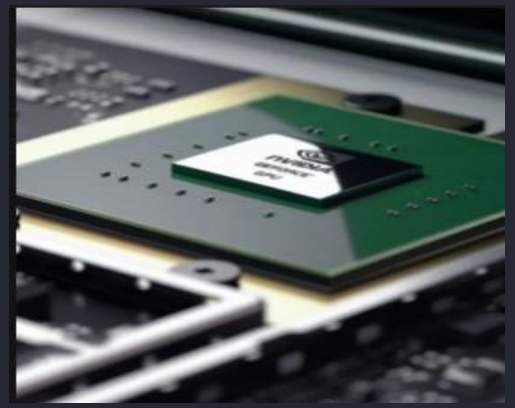
UNIVERSIDAD COOPERATIVA DE COLOMBIA



¿ Computación gráfica el impacto del Hardware ?







1. CPU (La Directora)

- **Rol:** Lógica, física del entorno e interacciones del usuario.
- **Operación gráfica:** Ejecuta el *Game Loop (lee entradas,actualiza pantalla y lee objetos)* y decide qué objetos ve la cámara (*Frustum Culling*).
- **Impacto en Software:** Envía las órdenes de dibujo a la GPU (*Draw Calls*). Su exceso satura la CPU.

2. GPU (La Fuerza Bruta)

- **Rol:** Procesamiento masivo de datos matemáticos en paralelo.
- **Operación gráfica:** Transforma las coordenadas 3D a la pantalla 2D y calcula el color de cada píxel (*Shaders*).
- **Impacto en Software:** Procesa millones de polígonos e iluminación simultáneamente en tiempo real.

3. Memoria (RAM vs. VRAM)

- **RAM (Sistema):** Almacena el código base y los archivos originales (.gltf, .png) al cargar la aplicación.
- **VRAM (Gráfica):** Memoria ultra rápida en la GPU. Almacena las mallas de vértices y texturas descomprimidas para acceso inmediato.
- **Impacto en Software:** La transferencia de datos (RAM ➡ VRAM) debe hacerse **una sola vez** al inicio para evitar caídas de fotogramas (FPS)

¿Qué es WebGL?



Web Graphics Library. Es la tecnología que permite a tu navegador usar la Tarjeta Gráfica (GPU) de tu PC para dibujar en 3D sin instalar nada extra.

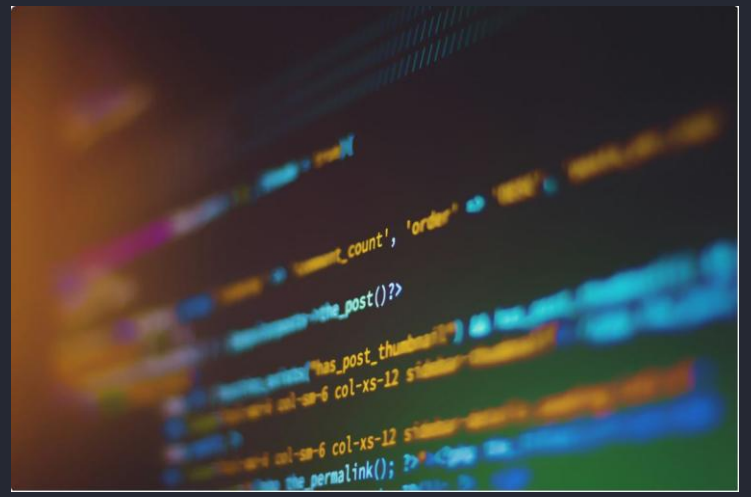
WebGL es la API estándar nativa del navegador para comunicarse con la tarjeta gráfica, pero el ecosistema se divide en tecnologías de bajo nivel (las que hablan con el hardware) y librerías de alto nivel (los motores que facilitan el trabajo, como Three.js).

Explicación Docente: "La CPU (procesador) es como un profesor de matemáticas muy inteligente, pero que resuelve un problema a la vez. La GPU es como un ejército de 1000 niños de primaria: no saben matemáticas complejas, pero pueden pintar millones de píxeles al mismo tiempo".

El Problema de WebGL

Escribir WebGL puro es un dolor de cabeza. Dibujar un simple triángulo requiere más de 100 líneas de código matemático complejo.

Explicación Docente: Aquí es donde entra nuestra herramienta salvadora: Three.js. Esta librería traduce nuestro lenguaje humano a las matemáticas complejas que requiere la GPU.



¿Qué es Three.js?



Three.js es una biblioteca o framework estándar de JavaScript que permite crear y mostrar gráficos en 3D directamente en el navegador web de forma sencilla

Capa sobre WebGL: Utiliza la tecnología WebGL (Web Graphics Library) por debajo, lo que permite usar la tarjeta gráfica (GPU) del usuario para acelerar los gráficos sin instalar programas extra.

Conceptos de un plató de cine: Funciona de manera similar a una escena de filmación donde configuras elementos básicos como la escena (el espacio virtual), las cámaras (el punto de vista), las luces (para iluminar) y los objetos o mallas con sus respectivos materiales y texturas.

La Trinidad del 3D

Para que exista un mundo 3D, siempre necesitamos 3 cosas:

La Escena (Scene)

El universo, el espacio vacío donde existirán los objetos, El contenedor global (espacio 3D) donde conviven todos los objetos, luces y cámaras.

La Cámara (Camera)

Nuestros ojos dentro de ese entorno, El punto de vista matemático que define la proyección (el paso de 3D a 2D)

El Renderizador (Renderer)

El motor que toma la foto de lo que ve la cámara y lo plasma en la pantalla plana de la computadora, El motor que toma la escena y la cámara para dibujar los píxeles en un elemento <canvas> de HTML5 a través de la GPU

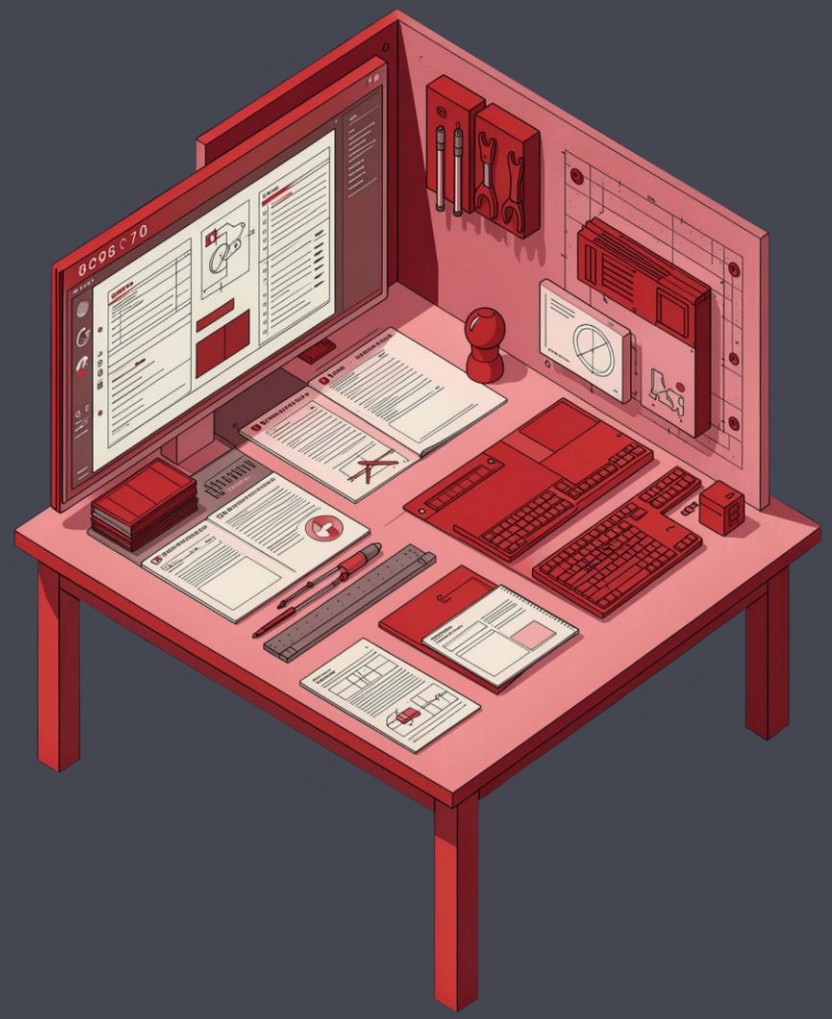
Analogía Hollywood: Imaginen un set de grabación. Necesitas el estudio vacío (Escena), el camarógrafo (Cámara) y el proyector del cine (Renderizador).

UNIVERSIDAD COOPERATIVA DE COLOMBIA

FASE 3

LAS HERRAMIENTAS

Configurando el entorno de desarrollo para nuestro primer mundo 3D



Herramientas a Utilizar

IDE: Visual Studio Code

Ya lo tienen instalado por clases previas. Nuestro editor principal de código.

Servidor Local: Live Server

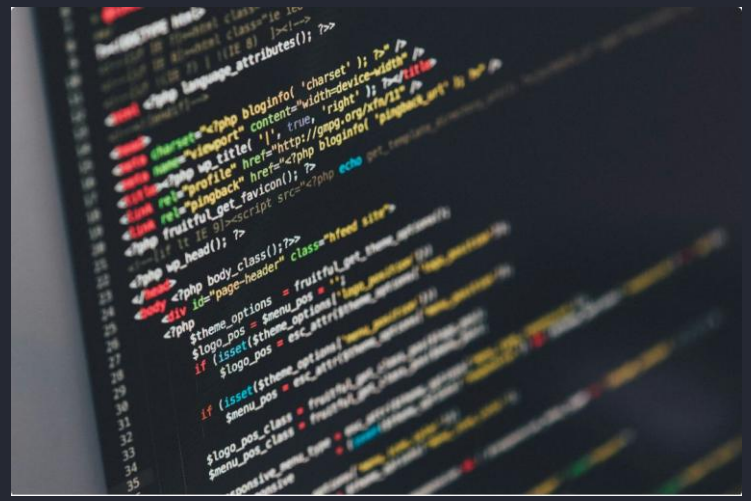
Extensión de VS Code que recarga automáticamente el navegador al guardar cambios.

Librería 3D: Three.js

Vía CDN / Enlace web. Sin instalación, directo desde la nube.

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Preparando el Entorno (*En Vivo*)

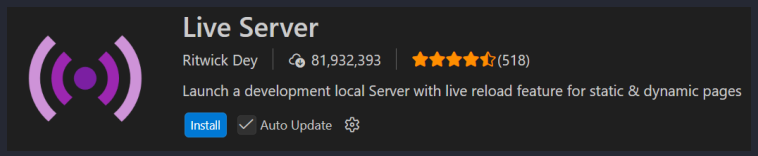


1 Abrir VS Code en Windows

2 Ir al panel de extensiones

Cuadritos a la izquierda de la barra lateral.

3 Buscar "Live Server"



Creada por Ritwick Dey → darle a "Instalar" (toma 5 segundos).

Explicación Docente: Esta herramienta nos creará un servidor local al instante para que nuestro código 3D se actualice cada vez que guardamos, sin tener que recargar la página manualmente.



FASE 4

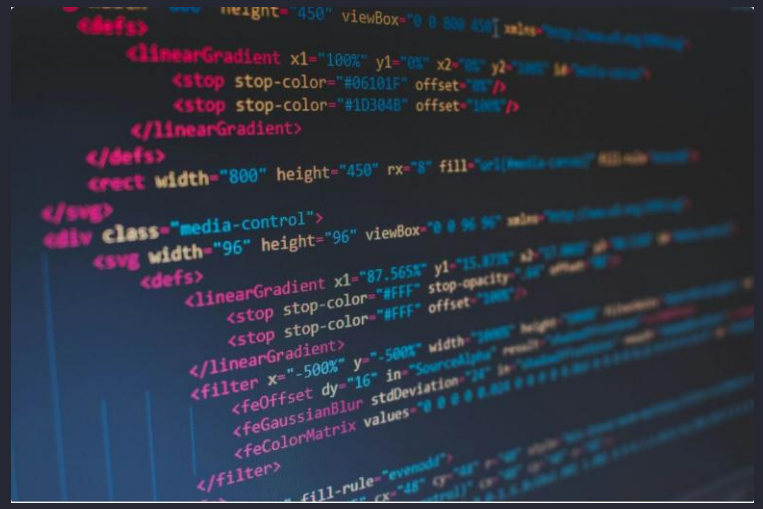
EL LABORATORIO

Creación de nuestro primer mundo y cubo 3D en WebGL con Three.js

UNIVERSIDAD COOPERATIVA DE COLOMBIA

Lab 1 – Creación de Estructura Base, importando la libreria Three.js

Crear archivo en una carpeta del escritorio laboratorio 2 septiembre y dentro el archivo index.html:



```
<!DOCTYPE html>
<html>
<head>
  <title>Mi Primer Mundo 3D</title>
  <style> body { margin: 0; overflow: hidden; } </style>
  <!-- Importación: Importamos Three.js con este enlace web , es una
   libreria de javascript para trabajar objetos y modelado de objetos en 3d-->
  <script
src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></scrip
t>
</head>
<body>
  <script src="app.js"></script>
</body>
</html>
```

Explicación Docente: El CSS (margin: 0) quita los bordes blancos del navegador. El script CDN nos ahorra descargar e instalar la librería. Todo nuestro trabajo se hará en app.js.

Lab 2 - Escena y Renderizador

Crear archivo app.js y escribir:

```
// 1. Creamos el Universo (Scene)
const scene = new THREE.Scene();
scene.background = new THREE.Color(0x111111);

// 2. Creamos el Motor de Renderizado
const renderer = new THREE.WebGLRenderer();
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);
```

Explicación Docente: Aquí preparamos el lienzo. setSize le dice al motor que queremos que nuestro mundo 3D ocupe todo el ancho y alto de la ventana del navegador.



Lab 3 - Posicionando la Cámara



Continuar en app.js:

```
// 3. Creamos la Cámara
const camera = new THREE.PerspectiveCamera(
  75,
  window.innerWidth / window.innerHeight,
  0.1,
  1000
);
camera.position.z = 5; // Alejamos la cámara hacia nosotros
```

Explicación Docente: La cámara imita al ojo humano (Perspective). Inicialmente, todo aparece en el centro (coordenada 0,0,0). Si dejamos la cámara en el centro, ¡no veremos nada porque estaríamos dentro del objeto! Por eso la retrocedemos 5 pasos en el eje Z.

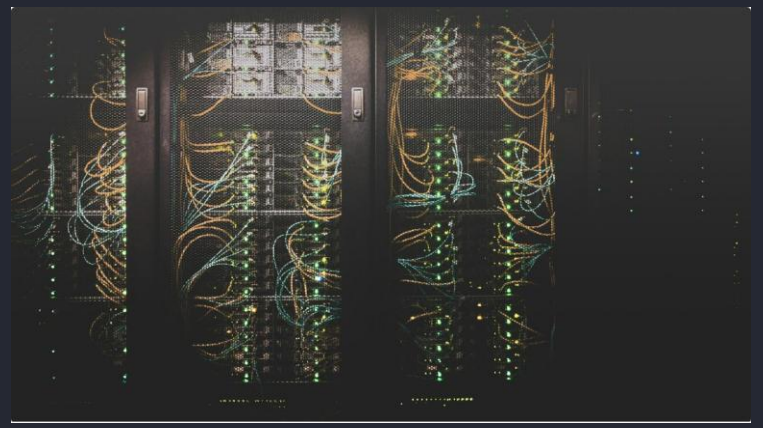
Lab 4 - Geometría y Material (Mesh)

Continuar en app.js:

```
// Creamos la forma matemática (Esqueleto)
const geometry = new THREE.BoxGeometry(1, 1, 1);

// Creamos la pintura/textura (Piel)
const material = new THREE.MeshBasicMaterial({ color: 0x00ffcc });

// Unimos Esqueleto + Piel = Malla (Mesh)
const cube = new THREE.Mesh(geometry, material);
scene.add(cube); // Lo agregamos al universo
```



Explicación Docente: En 3D, la forma (geometría) y el color (material) son cosas separadas. Juntarlas nos da un Mesh, que es el objeto real que podemos tocar y ver en nuestro mundo.

Lab 5 - Tomando la Foto (Render Final)

Al final de app.js:

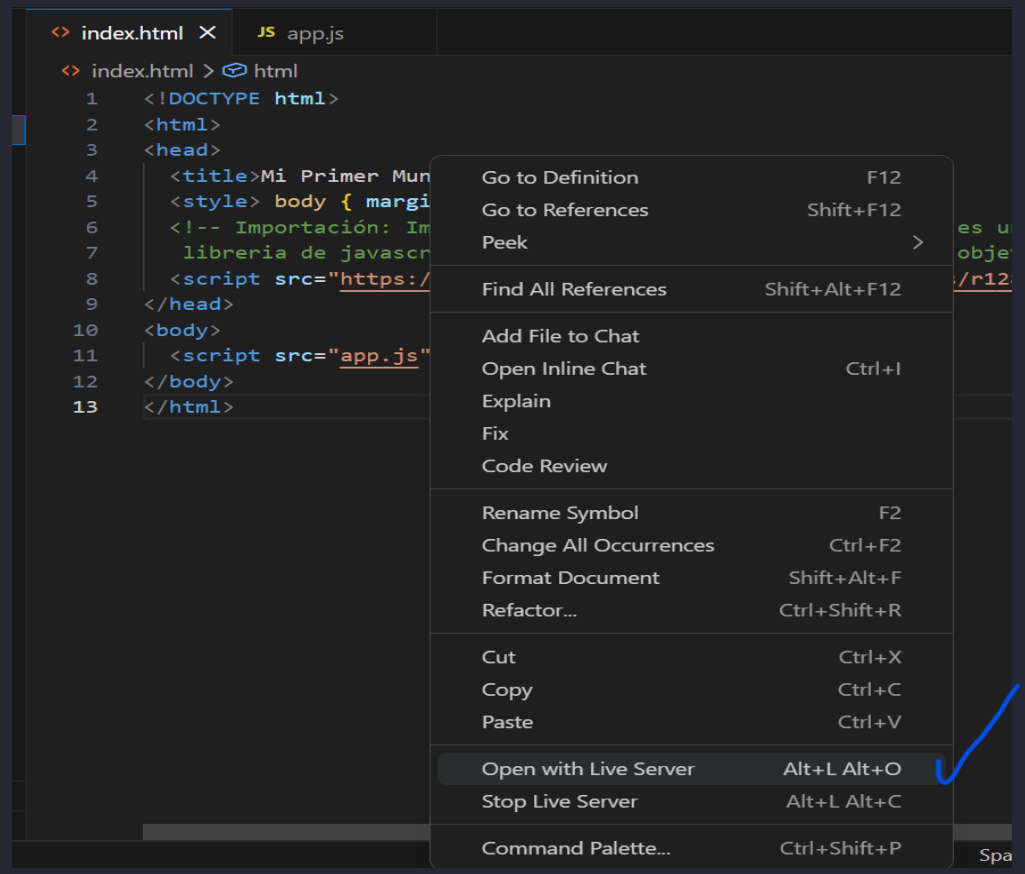
```
// El renderizador toma la foto de la escena
// desde los ojos de la cámara
renderer.render(scene, camera);
```

Explicación Docente: Ahora hacemos Click derecho en el archivo index.html → "Open with Live Server".

🎉 Resultado: En este punto verán un cuadrado cyan en su pantalla. (Pausa para que todos celebren y el docente ayude a quienes tengan errores tipográficos).

Lab 5 – Validando el Resultado

Ejecución: en App.js Ahora hacemos Click derecho en el archivo index.html → "Open with Live Server".



🎉 Resultado: En este punto verán un cuadrado cyan en su pantalla, sencillo pero es una muestra de grafico utilizando three.js y hacienda mas sencillo el uso de webgl con una libreria mucho mas amigable

Lab 6 - Dándole Vida (Animación)

En app.js que es el panel de control de nuestro grafico vemos que el resultado del dibujo de un cuadrado fijo aburre. Vamos a convertirlo en un cubo 3D animado usando un bucle.

Reemplazar la línea renderer.render... por esto:

```
function animar() {
  requestAnimationFrame(animar); // 60 veces por segundo
  cube.rotation.x += 0.01; // Gira en X
  cube.rotation.y += 0.01; // Gira en Y
  renderer.render(scene, camera);
}
animar(); // Arranca el motor
```

HASTA AQUI VAMOS CON EL EJERCICIO AHORA SI SE REQUIERE CAMBIAR FORMAS Y





Explicación Docente: Como en el cine, el movimiento 3D es una ilusión. Tomamos 60 fotos por segundo, y en cada foto, rotamos el cubo un grado. ¡Boom, tenemos animación fluida!

Reto de Clase

Cambiar el Color

Entra a Google, busca un color en formato hexadecimal (ej. #ff0055, pero cámbiale el # por 0x). Úsalo en MeshBasicMaterial.

Cambiar la Geometría

Cambia BoxGeometry() a TorusGeometry(1, 0.4, 16, 100) — ¡una dona 3D! Modifica también la velocidad de rotación.

Explicación Docente: La idea es que pierdan el miedo a romper el código, jueguen con las matemáticas y vean el resultado visual de inmediato.

FASE 5

LA EVIDENCIA Y EL RETO

Demostración del aprendizaje y reto de personalización



Lab 7 – Cambiar la forma Geometrica y dar realce

En app.js en la linea: `// Reemplaza tu línea de THREE.BoxGeometry por esta HEXAGONO:`

```
const geometry = new THREE.CylinderGeometry(1, 1, 1, 6);
```

Comentar las formas que no se quiera usar '//' aquí hay otras formas:

```
// Opcion 1: Cubo estándar
// const geometry = new THREE.BoxGeometry(1, 1, 1);

// Opcion 2: Cilindro o Hexágono (6 lados)
// const geometry = new THREE.CylinderGeometry(1, 1, 1, 6);

// Opcion 3: Esfera
// const geometry = new THREE.SphereGeometry(1, 16, 16);

// Opcion 4: Dona (Torus) - Esta está activa porque NO tiene '//'
const geometry = new THREE.TorusGeometry(0.8, 0.3, 16, 100);

const material = new THREE.MeshBasicMaterial({ color: 0x00ffcc });
const miObjeto = new THREE.Mesh(geometry, material);
scene.add(miObjeto);
```



Explicación Docente: Como en el cine, el movimiento 3D es una ilusión. Tomamos 60 fotos por segundo, y en cada foto, rotamos el cubo un grado. ¡Boom, tenemos animación fluida!

Lab 8 – Cambiar automaticamente la forma y animar

// debajo del bloque de set Interval

```
// Guardamos todas las formas en una lista
const listaDeFormas = [
    new THREE.BoxGeometry(1, 1, 1),             // 0: Cubo
    new THREE.CylinderGeometry(1, 1, 1, 6),     // 1: Hexágono
    new THREE.SphereGeometry(1, 16, 16),        // 2: Esfera
    new THREE.TorusGeometry(0.8, 0.3, 16, 100)  // 3: Dona
];

const material = new THREE.MeshBasicMaterial({ color: 0x00ffcc });

// Iniciamos con la forma 0 (Cubo)
let indice = 0;
const miObjeto = new THREE.Mesh(listaDeFormas[indice], material);
scene.add(miObjeto);

// Cambiamos la forma automáticamente cada 2 segundos
setInterval(() => {
    indice = (indice + 1) % listaDeFormas.length;
    miObjeto.geometry = listaDeFormas[indice]; // Actualiza el esqueleto
}, 2000);
```

//continua la Funcion Animar

- **El Catálogo (listaDeFormas)** Creamos una lista (Array) que funciona como un "menú" almacenando cuatro esqueletos matemáticos diferentes (cubo, hexágono, esfera y dona).
- **El Cambio Automático (setInterval):** Es un temporizador programado para ejecutarse cada 2 segundos (2000 milisegundos). Su trabajo es avanzar al siguiente elemento de la lista y reemplazar el esqueleto actual del objeto (miObjeto.geometry). El uso del operador módulo (%) es un truco matemático para que, al llegar al final de la lista, el ciclo vuelva a empezar desde cero.
- **El Motor de Vida (animar):** Es un ciclo infinito que le ordena al navegador tomar 60 "fotos" por segundo (requestAnimationFrame). En cada foto, el objeto gira una fracción de grado (rotation) y el renderizador actualiza la pantalla (renderer.render). Sin este ciclo, el objeto cambiaría de forma en la memoria de la computadora, pero nosotros nunca veríamos la actualización en la pantalla web.

Evidencia a Entregar



Instrucción: Toma una captura de pantalla donde se vea tu editor VS Code a la izquierda con el código fuente y el navegador a la derecha mostrando tu objeto (cubo o dona) renderizado y rotando.

Entrega: Súbela a la plataforma de la universidad en el espacio habilitado para esta semana.

Captura de pantalla

VS Code + Navegador visibles al mismo tiempo

Objeto personalizado

Tu color y/o geometría modificada

Subir a plataforma

En el espacio de entrega de la Semana 1

Conclusión e Integración

☑ Logro de Hoy

Hoy superaron la barrera técnica del 3D. Entendieron que configurar la cámara y la escena es el paso uno de cualquier metaverso.

🚀 Próxima Clase

A este mundo vacío le agregaremos leyes de la física (gravedad), luces realistas y cargaremos modelos 3D complejos exportados desde Blender. ¡El proyecto integrador final ya empezó a tomar forma!

